Agentic economy

Create your first ERC-8183 job

Copy page

Create an ERC-8183 job, fund escrow with USDC, submit a deliverable hash, and complete settlement on Arc Testnet.

This quickstart guides you through the ERC-8183 job lifecycle on Arc Testnet. You'll create developer-controlled smart contract account wallets, create a job, fund escrow with USDC, submit a deliverable hash, and complete the job as the evaluator. Select the tab that matches your preferred setup.

ERC-8183 contract on Arc testnet

Contract	Address
AgenticCommerce reference implementation	`0x0747EEf0706327138c69792bF28Cd525089e4583`

Circle Wallets Viem

Prerequisites

Before you begin, make sure you have:

1. A **Circle Developer Console** account
2. An API key created in the Console: **Keys → Create a key → API key → Standard Key**
3. Your **Entity Secret registered**

Step 1. Set up your project

Create a project directory, install dependencies, and configure your environment.

1.1. Create the project and install dependencies

Node.js Python

```
mkdir erc8183-quickstart
cd erc8183-quickstart
npm init -y
npm pkg set type=module
npm pkg set scripts.start="tsx --env-file=.env index.ts"

npm install @circle-fin/developer-controlled-wallets viem
npm install --save-dev tsx typescript @types/node
```

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Set environment variables

Create a `.env` file in the project directory and add your Circle credentials:

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
```

- `CIRCLE_API_KEY` is your Circle Developer API key.
- `CIRCLE_ENTITY_SECRET` is your registered Entity Secret.

The `npm run start` command loads variables from `.env` using Node.js native env-file support. The `python index.py` command loads the same `.env` file via `python-dotenv`.

Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Create developer-controlled wallets

In this step, you create two Arc Testnet dev-controlled wallets for the ERC-8183 flow: a client wallet and a provider wallet. In this quickstart, the client also acts as the evaluator. If you already have two Arc Testnet funded dev-controlled wallets for this flow, skip to **Step 4**.

The Step 2 through 9 sections explain the flow in smaller pieces. Not every step includes a code snippet, and the snippets are not cumulative. To run the full workflow end to end, use the **complete script** at the end of this tutorial.

index.ts index.py

```
const walletSet = await circleClient.createWalletSet({
  name: "ERC8183 Job Wallets",
});

const walletsResponse = await circleClient.createWallets({
  blockchains: ["ARC-TESTNET"],
  count: 2,
  walletSetId: walletSet.data?.walletSet?.id ?? "",
  accountType: "SCA",
});

const clientWallet = walletsResponse.data?.wallets?.[0]!;
const providerWallet = walletsResponse.data?.wallets?.[1]!;

console.log(`Client:   ${clientWallet.address} (${clientWallet.id})`);
console.log(`Provider: ${providerWallet.address} (${providerWallet.id})`);
console.log(`Evaluator: ${clientWallet.address} (${clientWallet.id})`);
```

Step 3. Fund the client wallet

The script will pause to allow you to fund the client wallet with Arc Testnet USDC from one of these faucets:

- Circle Faucet
- Circle Console Faucet

You only fund the client wallet as the script transfers starter USDC to the provider wallet automatically before the ERC-8183 flow begins.

> The public faucet is rate-limited, so this quickstart avoids requiring a second faucet request for the provider wallet.

Step 4. Create the job

Call `createJob(provider, evaluator, expiredAt, description, hook)` on the deployed ERC-8183 reference implementation. This creates the job in the `Open` state. This quickstart

uses `address(0)` for `hook` so the flow stays on the default non-hooked path.

index.ts index.py

```
const createJobTx = await circleClient.createContractExecutionTransaction({
  walletAddress: clientWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: AGENTIC_COMMERCE_CONTRACT,
  abiFunctionSignature: "createJob(address,address,uint256,string,address)",
  abiParameters: [
    providerWallet.address!,
    clientWallet.address!,
    expiredAt.toString(),
    "ERC-8183 demo job on Arc Testnet",
    "0x0000000000000000000000000000000000000000",
  ],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});
```

Step 5. Set the budget

In this deployed contract, the provider sets the job price by calling `setBudget(jobId, amount, optParams)`.

index.ts index.py

```
const setBudgetTx = await circleClient.createContractExecutionTransaction({
  walletAddress: providerWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: AGENTIC_COMMERCE_CONTRACT,
  abiFunctionSignature: "setBudget(uint256,uint256,bytes)",
  abiParameters: [jobId.toString(), JOB_BUDGET.toString(), "0x"],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});
```

Step 6. Approve USDC and fund escrow

Before the client can fund the job, the USDC contract must approve the ERC-8183 contract to transfer the escrow amount. Then the client calls `fund(jobId, optParams)` to move the job into the `Funded` state.

index.ts index.py

```
const approveTx = await circleClient.createContractExecutionTransaction({
  walletAddress: clientWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: "0x3600000000000000000000000000000000000000",
  abiFunctionSignature: "approve(address,uint256)",
  abiParameters: [AGENTIC_COMMERCE_CONTRACT, JOB_BUDGET.toString()],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});

const fundTx = await circleClient.createContractExecutionTransaction({
  walletAddress: clientWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: AGENTIC_COMMERCE_CONTRACT,
  abiFunctionSignature: "fund(uint256,bytes)",
  abiParameters: [jobId.toString(), "0x"],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});
```

Step 7. Submit the deliverable

The provider submits a `bytes32` deliverable hash, moving the job into the `Submitted` state.

index.ts index.py

```
const deliverableHash = keccak256(toHex("arc-erc8183-demo-deliverable"));

const submitTx = await circleClient.createContractExecutionTransaction({
  walletAddress: providerWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: AGENTIC_COMMERCE_CONTRACT,
  abiFunctionSignature: "submit(uint256,bytes32,bytes)",
  abiParameters: [jobId.toString(), deliverableHash, "0x"],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});
```

Step 8. Complete the job

The evaluator completes the job by calling `complete(jobId, reason, optParams)`. In this quickstart, the client is also the evaluator.

index.ts index.py

```
const reasonHash = keccak256(toHex("deliverable-approved"));

const completeTx = await circleClient.createContractExecutionTransaction({
  walletAddress: clientWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: AGENTIC_COMMERCE_CONTRACT,
  abiFunctionSignature: "complete(uint256,bytes32,bytes)",
  abiParameters: [jobId.toString(), reasonHash, "0x"],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});
```

Step 9. Check the final job state

Read the job back from the contract to confirm it reached `Completed`. This reference implementation does not return the deliverable in `getJob()`, so the script prints the submitted deliverable hash from local flow state instead.

index.ts index.py

```
const job = await publicClient.readContract({
  address: AGENTIC_COMMERCE_CONTRACT,
  abi: agenticCommerceAbi,
  functionName: "getJob",
  args: [jobId],
});

console.log(`Job ID: ${jobId}`);
console.log(`Status: ${STATUS_NAMES[Number(job.status)]}`);
console.log(`Budget: ${formatUnits(job.budget, 6)} USDC`);
console.log(`Hook: ${job.hook}`);
console.log(`Deliverable hash submitted: ${deliverableHash}`);
```

Full job lifecycle script

These complete scripts below combines all the preceding steps into a single runnable file.

index.ts index.py

```
import { createInterface } from "node:readline/promises";
import { setTimeout as delay } from "node:timers/promises";
import { stdin as input, stdout as output } from "node:process";
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/develope
import {
  createPublicClient,
  decodeEventLog,
```

See all 354 lines

Run the script:

Node.js Python

```
npm run start
```

Verify the result

If the flow succeeds, the output should show:

- a created job ID
- a completed final status
- the client balance reduced by the funded escrow amount
- the provider balance increased after completion

> If platform or evaluator fees are configured on the deployed contract, the provider receives the net amount after fees rather than the full job budget.

You can also inspect the transaction links in the terminal output on **Arcscan Testnet**.

Summary

After completing this quickstart, you've successfully:

- Set up a project for running an ERC-8183 job flow on Arc Testnet
- Prepared client and provider wallets for the client, provider, and evaluator roles
- Walked through an example ERC-8183 job lifecycle
- Confirmed balances and job state in the script output and reviewed transactions on Arcscan Testnet

Was this page helpful? Yes No

Register your first AI agent App Kit overview

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.